Finetuning

When to change weights, memory math, numerical formats, LoRA, model merging, and training tactics

Fine-tuning is a targeted behavior change, not the default solution to missing information. Establish evaluation and simpler baselines before spending on data and training.

Source: Chip Huyen, AI Engineering: Building Applications with Foundation Models, first edition (2025). Source: book pp. 307–362 / PDF p. 331. These are original study notes, not verbatim chapter reproductions. Tool names, prices, model comparisons, and research findings reflect the book's publication period unless stated otherwise.

01 Finetuning Overview and When to Use It

Source: book pp. 308–319 / PDF p. 332

Transfer learning and training objectives

Fine-tuning starts from trained weights and adapts some or all parameters. Transfer learning improves sample efficiency because general capabilities need not be learned again. Continued pre-training uses self-supervised sequences; supervised instruction tuning uses demonstrations; preference tuning uses comparative feedback.

Choose an objective suited to the task. Infilling trains completion of missing spans; instruction tuning shapes response behavior. A released instruction model has already been adapted, so evaluate its existing behavior before continuing training.

Reasons to fine-tune and reasons not to

Good candidates include consistent domain behavior, output style/schema, niche syntax or SQL dialects, specialized classification, bias mitigation, or distillation into a smaller serving model. Moving repeated examples into weights can shorten prompts, though prompt caching changes the economic comparison.

Reasons to pause: inadequate evaluation, insufficient trustworthy data, unresolved prompting/retrieval errors, maintenance burden, hosting constraints, or a stronger base model that already solves the task. Fine-tuning can overfit, amplify data defects, or cause catastrophic forgetting of other capabilities. Keep broad regression tests alongside the target task.

Fine-tuning and RAG solve different gaps

RAG chiefly addresses information availability and freshness; fine-tuning chiefly addresses how to perform a task or respond. Weight updates are not a dependable database-update mechanism. A tuned model can still need retrieval, and tuning can improve how it uses retrieved evidence.

Suggested sequence: evaluate a strong base with good prompts, diagnose whether missing knowledge is the issue, try a simple retrieval baseline if it is, then tune when a persistent behavior gap remains. Compare costs and task quality at each stage. Neither RAG nor fine-tuning is categorically superior.

---

02 Memory Bottlenecks and Memory Math

Source: book pp. 319-325 / PDF p. 343

Forward/backward passes and trainable parameters

Inference performs forward computation. Training also retains information for gradients, computes a backward pass, and updates parameters with an optimizer. Frozen weights still occupy memory and participate in forward computation, but do not require optimizer updates.

> Training memory = weights + activations + gradients + optimizer state + runtime overhead
> Inference memory = weights + activations/workspace + KV cache + runtime overhead

Activation memory grows with batch size, sequence length, architecture, and implementation. Reducing trainable parameters does not remove every term. Some implementations use higher-precision master weights and optimizer state, which must be included explicitly.

Worked storage estimate

> 7 billion weights × 2 bytes = 14 GB of weight storage
> 7 billion weights × 0.5 bytes = 3.5 GB at ideal 4-bit packing

These are decimal GB and exclude quantization metadata, caches, activations, and framework overhead. A device with nominal capacity equal to checkpoint size is not enough.

With Adam-like optimization, a trainable parameter can have a gradient and two optimizer-state values, sometimes held in different precision. Work out each term's byte width rather than assuming a universal multiplier. PEFT reduces gradient/optimizer storage; activation checkpointing trades recomputation for lower activation storage. Hardware sizing needs workload-level measurement.

03 Numerical Representations and Quantization

Source: book pp. 325-332 / PDF p. 349

Precision, range, and format compatibility

Floating-point formats divide bits among sign, exponent, and significand. More exponent bits provide a wider representable range; more significand bits provide finer precision. FP32 uses 4 bytes; FP16 and BF16 use 2 bytes each, but BF16 preserves a wider range with less fine precision than FP16.

TF32 is an NVIDIA computation format with reduced mantissa precision, not a promise that all stored tensors use a 32-bit-equivalent effective precision. Loading a checkpoint into the wrong format can overflow or degrade it. Respect the model's intended dtype and hardware support.

What to quantize and when

Quantization maps values to a lower-bit representation. Weight, activation, gradient, and KV-cache quantization address different bottlenecks. Post-training quantization (PTQ) converts an existing model, often using calibration. Quantization-aware training (QAT) accounts for quantization effects while learning.

Mixed-precision training keeps sensitive calculations in higher precision while reducing others. Lower bits reduce storage, but speed gains depend on kernel/hardware support and conversion overhead. Measure task accuracy, outliers, and actual latency; aggressively quantized models can fail differently even when average benchmark scores look acceptable.

04 Parameter-Efficient Finetuning

Source: book pp. 332-347 / PDF p. 356

Partial fine-tuning, adapters, and soft prompts

Partial fine-tuning updates selected original layers. Adapter-based methods add a small set of learnable parameters while freezing most base weights. Soft prompt/prefix approaches learn continuous input or intermediate representations rather than a human-readable hard prompt.

Fewer trainable parameters reduce update-state memory and can improve data efficiency. They do not automatically reduce the base model's serving size. Adapter placement, task complexity, base capability, and training data govern results. Some adapters add inference computation; others can be merged.

LoRA: learn a low-rank weight update

For a base matrix W with shape $d_{out} \times d_{in}$, learn B of shape $d_{out} \times r$ and A of shape $r \times d_{in}$, while W remains frozen.

> $W' = W + (\alpha/r)BA$
> New trainable parameters = $r(d_{in} + d_{out})$
> Full matrix parameters = $d_{in}d_{out}$

Example: a 4096 × 4096 matrix has 16,777,216 weights. Rank 8 adds 65,536 parameters, about 0.39% of that matrix's size. LoRA learns the update, not a replacement factorization of W. Rank and target matrices jointly determine capacity and budget.

LoRA targets, serving, and QLoRA

Attention projections are common targets; feedforward projections can also matter. Under a fixed parameter budget, spreading rank across matrices can outperform concentrating it, but validate for the task. Higher rank increases capacity and memory without guaranteeing improvement.

Merge adapters into a compatible base for a single deployed model without the separate adapter computation, or keep them separate for many customer/task adapters sharing one base. The latter saves storage and switching cost but can add runtime complexity.

QLoRA stores frozen base weights in low precision, commonly 4-bit, and trains adapters with suitable compute precision. It further reduces memory but does not mean all activations, gradients, or calculations run in 4 bits. Include dequantization and optimizer/runtime overhead in estimates.

---

05 Model Merging and Multi-Task Finetuning

Source: book pp. 347-357 / PDF p. 371

Multi-task strategies and ensembling

Joint training mixes examples for all tasks; sequential tuning is simpler to stage but risks forgetting earlier tasks. Separate specialized models improve isolation but increase serving complexity. Merging aims to combine useful learned behavior into fewer deployed models.

Ensembling combines outputs from multiple models and pays multiple inference costs. Merging combines parameters or modules. Success requires compatible representations, architectures, and training histories; a simple arithmetic merge is not a universal skill-combination operation.

Summing, interpolation, task vectors, and pruning

Weighted averaging is simple for aligned compatible weights. SLERP interpolates along a spherical path rather than a straight line. Task arithmetic uses the difference between tuned and base weights as a task vector.

> Task vector $\tau_i = W_{tuned,i} - W_{base}$
> Candidate merge = $W_{base} + \Sigma \lambda_i \tau_i$

TIES/DARE-style methods reduce interference from redundant or conflicting task updates before merging. Such pruning concerns task vectors and does not necessarily shrink the deployed dense model. Evaluate every contributing task and safety behavior after merging.

Layer stacking and concatenation

Layer stacking can combine or upscale depth, followed by additional training to make the result work coherently. Concatenating modules or LoRA factors preserves separate capacity but increases dimensions/rank and may retain much of the original storage/computation cost.

Merging can be attractive for edge deployment or several related skills, but the apparent convenience does not remove validation. Keep base/adapters, merge recipes, licenses, and evaluation history for reproducibility and rollback.

---

06 Finetuning Tactics

Source: book pp. 357-362 / PDF p. 381

Frameworks, models, and controlled experiments

Validate the pipeline on a cheap small model, including saving, checkpoint loading, and evaluation, before launching expensive runs. Establish a strong reference model and then compare smaller candidates to map the quality/cost frontier.

Managed fine-tuning APIs reduce setup but may restrict data formats, training knobs, model versions, and deployment. Local frameworks give control at the cost of hardware and expertise. Check supported objectives, architecture, dtype, adapter targets, monitoring, and reproducibility.

Hyperparameters and loss masking

Learning rate, schedule, batch size, epochs, and adapter settings determine adaptation. Too-large steps can destroy useful behavior; too-small steps fail to adapt. More epochs can overfit. Larger effective batches smooth updates but require more memory.

Gradient accumulation combines microbatch gradients to simulate a larger update batch; it reduces peak storage but is not identical to physically parallel processing or all batch-dependent behaviors. Set prompt/response loss weights intentionally: instruction tuning often emphasizes assistant-response tokens, while user/system tokens may be masked or downweighted. Preserve model chat templates and evaluate held-out examples during training.

Revision Check

1. Is your failure caused by missing information or response behavior?
2. Which memory terms remain when the base weights are frozen?
3. How many trainable parameters does a LoRA adapter add?
4. When would separate adapters beat merged weights operationally?
5. How will you detect catastrophic forgetting?

The lowest-memory training method is not automatically the best system. Measure target gains, general regressions, data needs, deployment cost, and maintenance together.

Book-based notes, separate from the earlier transcript-based notes. Coverage is indexed by section in coverage.md. Source links open the supplied book at the relevant PDF page. Detailed examples, citations, and complete experimental evidence remain in the book.